FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Christian Dave K. Salvador
Section:	AN42
Professor:	Ms. Crisola Tan
Date:	Oct 04, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

SOURCE CODE:

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition --------
class RandomWalker(ap.Agent):

    def setup(self):
        self.position = (0, 0)
        self.path = []

    def step(self):
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        random.shuffle(directions)

        x, y = self.position

        # Try to move to a new position
        for dx, dy in directions:
            new_x = x + dx
            new_y = y + dy

            # Keep agents inside the grid
            if (0 <= new_x < self.model.p.grid_size[0] and
```

```
                    0 <= new_y < self.model.p.grid_size[1]):

                new_position = (new_x, new_y)

                # Avoid other agents
                occupied = False

                for other in self.model.agents:
                        if other != self and other.position ==
new_position:
                            occupied = True
                            break

                if not occupied:
                    self.position = new_position
                    break

        # Record the position in the path
        self.path.append(self.position)

# -------- Model Definition --------
class RandomWalkModel(ap.Model):

    def setup(self):

        # Create agents
        self.agents = ap.AgentList(
            self,
            self.p.agents,
            RandomWalker
        )

        # Initialize random positions
        positions = []

        for agent in self.agents:

            # Find an unoccupied starting position
            while True:
                position = (
                    random.randint(0, self.p.grid_size[0] - 1),
                    random.randint(0, self.p.grid_size[1] - 1)
                )

                if position not in positions:
```

```
                    positions.append(position)
                    break

            agent.position = position
            agent.path = [position]

        # Create grid
        self.grid = ap.Grid(
            self,
            self.p.grid_size,
            torus=False
        )

        self.grid.add_agents(self.agents)

    def step(self):

        for agent in self.agents:
            agent.step()

# -------- Parameters from user input --------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model --------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation --------
fig, ax = plt.subplots()

ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)

ax.set_xticks(range(model.p.grid_size[0]))
ax.set_yticks(range(model.p.grid_size[1]))

ax.grid(True)

# -------- Create lines for agent paths --------
path_lines = []
```

```
for agent in model.agents:
    line, = ax.plot([], [], linewidth=1)
    path_lines.append(line)

# -------- Create agent dots --------
scat = ax.scatter([], [], s=200)

# -------- Animation Update --------
def update(frame):

    # Move agents
    model.step()

    # Update agent positions
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]

    scat.set_offsets(list(zip(x, y)))

    # Update paths
    for i, agent in enumerate(model.agents):

        path_x = [position[0] for position in agent.path]
        path_y = [position[1] for position in agent.path]

        path_lines[i].set_data(path_x, path_y)

    return [scat] + path_lines

# -------- Run Animation --------
ani = animation.FuncAnimation(
    fig,
    update,
    frames=model.p.steps,
    blit=True,
    repeat=False
)

plt.title("Random Walker Simulation with Agent Paths")
plt.xlabel("X Position")
plt.ylabel("Y Position")

plt.show()

# -------- Display Final Positions --------
```

```
print("\nFinal Positions of Agents:")

for i, agent in enumerate(model.agents):
    print(
        f"Agent {i + 1}: "
        f"({agent.position[0]}, {agent.position[1]})"
    )

# -------- Analyze Distribution --------
print("\nFinal Position Distribution:")

position_count = {}

for agent in model.agents:

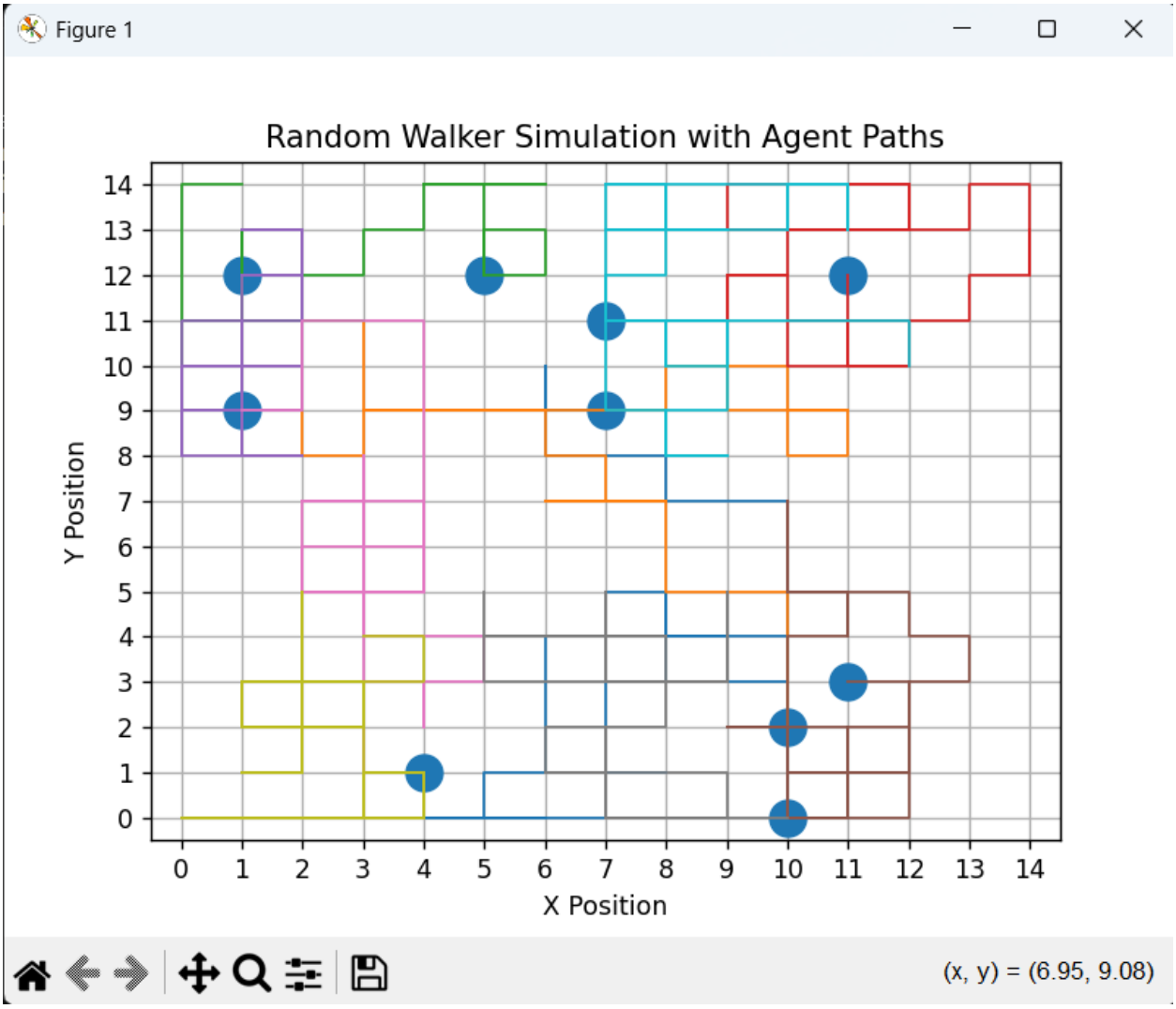
    position = agent.position

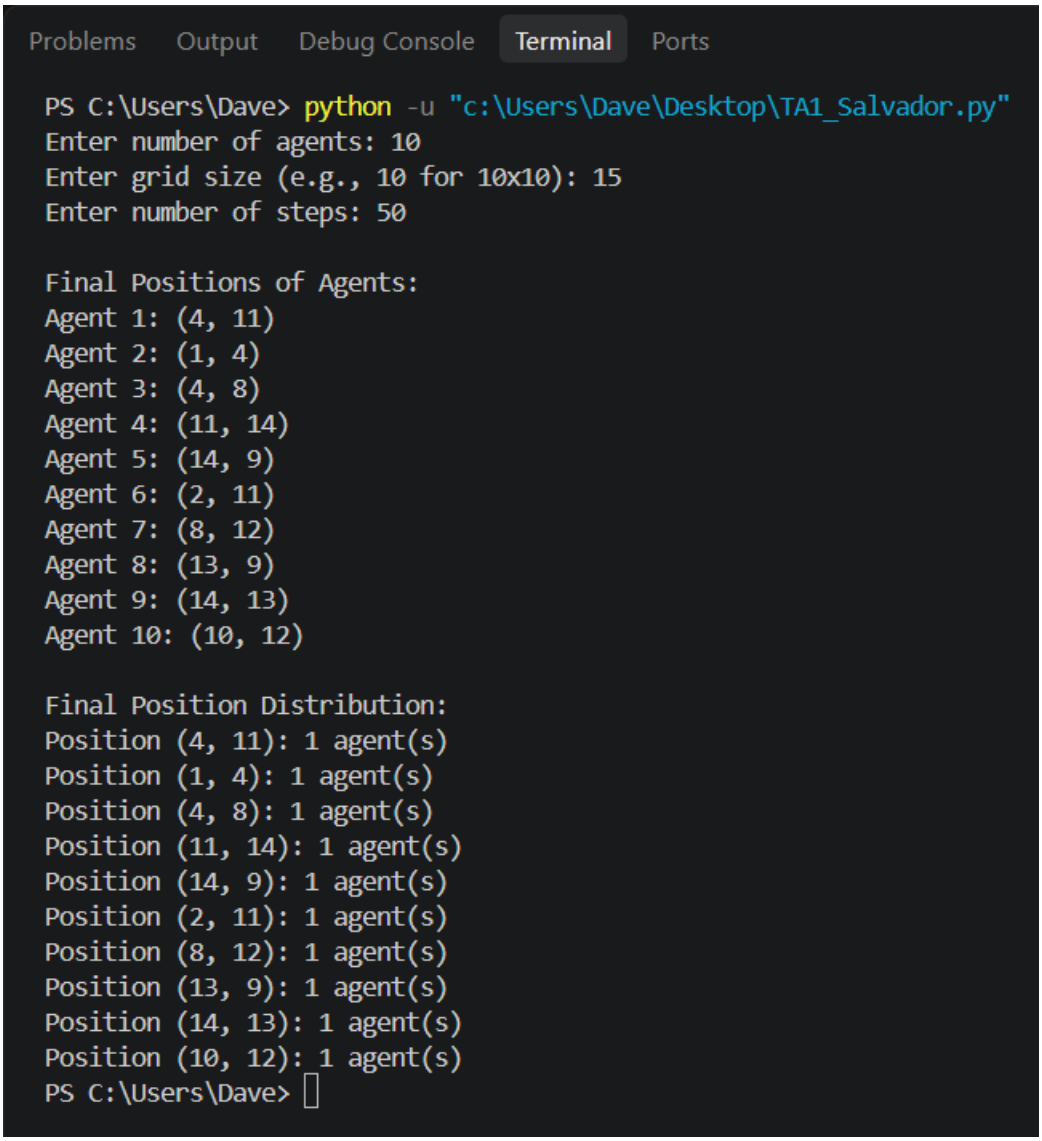
    if position in position_count:
        position_count[position] += 1
    else:
        position_count[position] = 1

for position, count in position_count.items():

    print(
        f"Position {position}: "
        f"{count} agent(s)"
    )
```

OUTPUT:


Figure 1
Random Walker Simulation with Agent Paths
Y Position
X Position
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
(x, y) = (6.95, 9.08)


Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\Dave> python -u "c:\Users\Dave\Desktop\TA1_Salvador.py"
Enter number of agents: 10
Enter grid size (e.g., 10 for 10x10): 15
Enter number of steps: 50
Final Positions of Agents:
Agent 1: (4, 11)
Agent 2: (1, 4)
Agent 3: (4, 8)
Agent 4: (11, 14)
Agent 5: (14, 9)
Agent 6: (2, 11)
Agent 7: (8, 12)
Agent 8: (13, 9)
Agent 9: (14, 13)
Agent 10: (10, 12)
Final Position Distribution:
Position (4, 11): 1 agent(s)
Position (1, 4): 1 agent(s)
Position (4, 8): 1 agent(s)
Position (11, 14): 1 agent(s)
Position (14, 9): 1 agent(s)
Position (2, 11): 1 agent(s)
Position (8, 12): 1 agent(s)
Position (13, 9): 1 agent(s)
Position (14, 13): 1 agent(s)
Position (10, 12): 1 agent(s)
PS C:\Users\Dave>

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. **If the number of agents is very high compared to the grid size, the agents will have less space to move around. Some agents may get blocked by others because they cannot occupy the same position. This can make the movement more crowded and limited.**
2. **Increasing the number of steps gives the agents more time to move from their starting positions. Their paths become longer, and they can spread to different parts of the grid. The longer they move, the more areas they can explore.**
3. **No, the agents do not move outside the grid because the program checks the grid boundaries before every movement. If a possible position is outside the grid, the agent will not move there. This keeps all agents within the given area.**
4. **This simulation can model real-life phenomena such as particle diffusion, crowd movement, or animals searching for food. It shows how individuals can move randomly while being affected by space and other individuals. This can help us understand how movement and distribution happen in real-life situations.**